SAÉ S3.A.01 - Développement d'une Application

Sprint 4 - Applications à d'autres suites complexes

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS

Sprint 3 - Rétrospective

Assurez-vous de toujours avoir la dernière version du sujet (la date de dépôt apparaît sur Moodle)

Sprint 3 - Rétrospective

Assurez-vous de toujours avoir la dernière version du sujet (la date de dépôt apparaît sur Moodle)

C'est assez difficile pour nous de tenir les délais pour les validations techniques et les démonstrations, mais on fait de notre mieux

Sprint 3 - Rétrospective

Assurez-vous de toujours avoir la dernière version du sujet (la date de dépôt apparaît sur Moodle)

C'est assez difficile pour nous de tenir les délais pour les validations techniques et les démonstrations, mais on fait de notre mieux

Et vous, avez-vous des remarques sur le sprint précédent ?

But du *sprint*

Vous devez pouvoir générer des fractales à partir de suites complexes autres que la suite de Julia, en utilisant la même approche que pour la suite de Julia

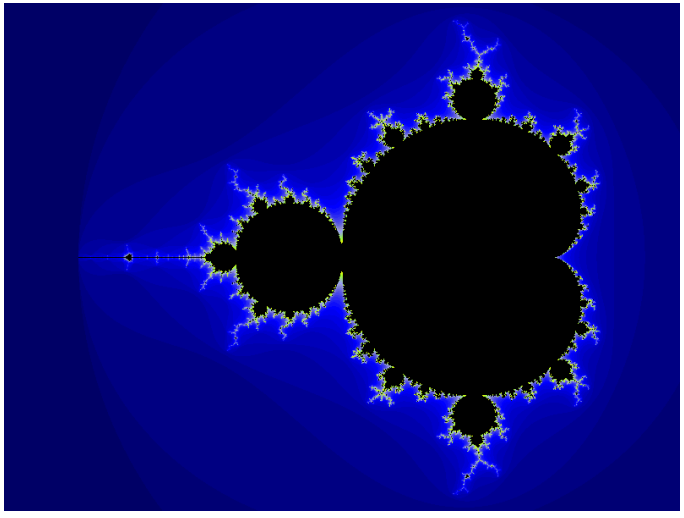


Figure – Fractale de Mandelbrot

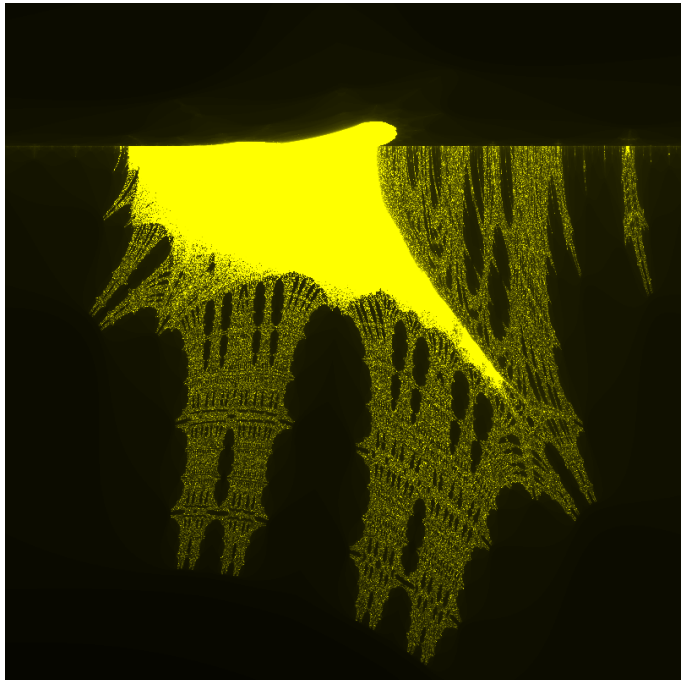


Figure – Fractale du *Burning-Ship*

Principe général

Pour proposer d'autres suites complexes, il vous suffit d'implémenter l'interface `IComplexSequence` dans une nouvelle classe pour chaque suite complexe demandée

Principe général

Pour proposer d'autres suites complexes, il vous suffit d'implémenter l'interface `IComplexSequence` dans une nouvelle classe pour chaque suite complexe demandée

Si vous avez bien respecté les consignes sur l'utilisation des interfaces dans votre code, vous n'aurez rien à modifier (à part votre `main`)

Ensemble de Mandelbrot

Il faut ici reprendre la même formule que celle de la suite de Julia, en remplaçant la constante c par le nombre complexe pour lequel la suite est évaluée :

$$f_m(z) = z^2 + z_0$$

Ensembles de Multijulia et Multibrot

Ces suites sont des généralisations de l'ensemble de Julia et de l'ensemble de Mandelbrot, en utilisant une puissance p plus élevée que 2 dans la formule de la suite :

$$f_j(z) = z^p + c$$

$$f_m(z) = z^p + z_0$$

Ensembles de Multijulia et Multibrot

Ces suites sont des généralisations de l'ensemble de Julia et de l'ensemble de Mandelbrot, en utilisant une puissance p plus élevée que 2 dans la formule de la suite :

$$f_j(z) = z^p + c$$

$$f_m(z) = z^p + z_0$$

Pour simplifier le code, il est préférable d'ajouter l'opération de puissance dans l'interface `IComplex`, et l'implémenter dans la classe `Complex`

Ensembles du *Burning Ship* et du *Tricorn*

Pour le *Burning Ship*, on reprend la formule de la suite de Julia en prenant la valeur absolue de la partie réelle et de la partie imaginaire du nombre complexe, avant d'élever au carré le nombre complexe obtenu :

$$f_b(z) = (|\mathfrak{Re}(z)| + i|\mathfrak{Im}(z)|)^2 + c$$

Ensembles du *Burning Ship* et du *Tricorn*

Pour le *Burning Ship*, on reprend la formule de la suite de Julia en prenant la valeur absolue de la partie réelle et de la partie imaginaire du nombre complexe, avant d'élever au carré le nombre complexe obtenu :

$$f_b(z) = (|\mathfrak{Re}(z)| + i|\mathfrak{Im}(z)|)^2 + c$$

Pour le *Tricorn*, on reprend la formule de la suite de Julia en prenant le conjugué du nombre complexe, avant de l'élever au carré :

$$f_t(z) = \overline{(z)}^2 + c$$

Ensembles du *Burning Ship* et du *Tricorn*

Pour le *Burning Ship*, on reprend la formule de la suite de Julia en prenant la valeur absolue de la partie réelle et de la partie imaginaire du nombre complexe, avant d'élever au carré le nombre complexe obtenu :

$$f_b(z) = (|\mathfrak{Re}(z)| + i|\mathfrak{Im}(z)|)^2 + c$$

Pour le *Tricorn*, on reprend la formule de la suite de Julia en prenant le conjugué du nombre complexe, avant de l'élever au carré :

$$f_t(z) = \overline{(z)}^2 + c$$

On peut aussi en faire des suites comme celle de Mandelbrot, en remplaçant c par z_0, ou les généraliser à des puissances plus élevées

Suites générales

Un point important dans la programmation orientée objet est de pouvoir réutiliser le code existant (vous avez normalement remarqué que le sujet est rédigé dans ce sens)

Suites générales

Un point important dans la programmation orientée objet est de pouvoir réutiliser le code existant (vous avez normalement remarqué que le sujet est rédigé dans ce sens)

Dans les classes précédentes, on est obligé à chaque fois de définir une classe pour chaque suite complexe, ce qui est assez lourd juste pour écrire une formule mathématique

Suites générales

Un point important dans la programmation orientée objet est de pouvoir réutiliser le code existant (vous avez normalement remarqué que le sujet est rédigé dans ce sens)

Dans les classes précédentes, on est obligé à chaque fois de définir une classe pour chaque suite complexe, ce qui est assez lourd juste pour écrire une formule mathématique

On peut à la place définir une implémentation générique de l'interface `IComplexSequence`, qui prendra en paramètre une fonction f définissant la suite complexe à générer, et qui l'utilisera pour calculer le terme suivant

Suites générales

Un point important dans la programmation orientée objet est de pouvoir réutiliser le code existant (vous avez normalement remarqué que le sujet est rédigé dans ce sens)

Dans les classes précédentes, on est obligé à chaque fois de définir une classe pour chaque suite complexe, ce qui est assez lourd juste pour écrire une formule mathématique

On peut à la place définir une implémentation générique de l'interface `IComplexSequence`, qui prendra en paramètre une fonction f définissant la suite complexe à générer, et qui l'utilisera pour calculer le terme suivant

On va donc réutiliser une expression lambda

Représentation d'une formule à l'aide d'une expression lambda

Reprenons par exemple la suite de Mandelbrot, qui est définie par la formule suivante :

$$f_m(z) = z^2 + z_0$$

Représentation d'une formule à l'aide d'une expression lambda

Reprenons par exemple la suite de Mandelbrot, qui est définie par la formule suivante :

$$f_m(z) = z^2 + z_0$$

On peut l'écrire en Java à l'aide d'une expression lambda :

```
(z, z0) -> z.multiply(z).add(z0)
```

Représentation d'une formule à l'aide d'une expression lambda

Reprenons par exemple la suite de Mandelbrot, qui est définie par la formule suivante :

$$f_m(z) = z^2 + z_0$$

On peut l'écrire en Java à l'aide d'une expression lambda :

```
(z, z0) -> z.multiply(z).add(z0)
```

Cette expression lambda est du type `BinaryOperator<IComplex>`, qui définit une méthode `apply` prenant deux paramètres de type `IComplex` et retournant un résultat de type `IComplex`

Attendus de ce *sprint*

- Implémentation de la suite de Mandelbrot
- Implémentation de la suite de Julia généralisée
- Implémentation de la suite de Mandelbrot généralisée
- Implémentation de la suite *Burning Ship*
- Implémentation de la suite *Burning Ship* généralisée
- Implémentation de la suite *Tricorn*
- Implémentation de la suite *Tricorn* généralisée
- Implémentation d'au moins une suite complexe définie à l'aide d'une expression plus générale utilisant une expression lambda
- Tests unitaires associés à ces fonctionnalités

Validations techniques à réaliser en cours de *sprint*

- Implémentation de la suite de Julia généralisée
- Implémentation de la suite de Mandelbrot généralisée
- Utilisation des expressions lambda pour les suites générales

Démonstration finale

Vous devez pouvoir générer des fractales à partir de suites complexes autres que la suite de Julia, en utilisant la même approche que pour la suite de Julia, en utilisant des palettes de couleurs au choix

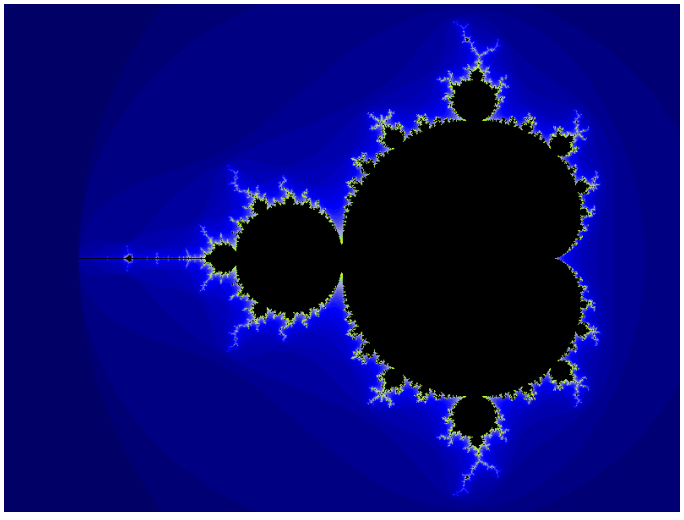


Figure – Fractale de Mandelbrot

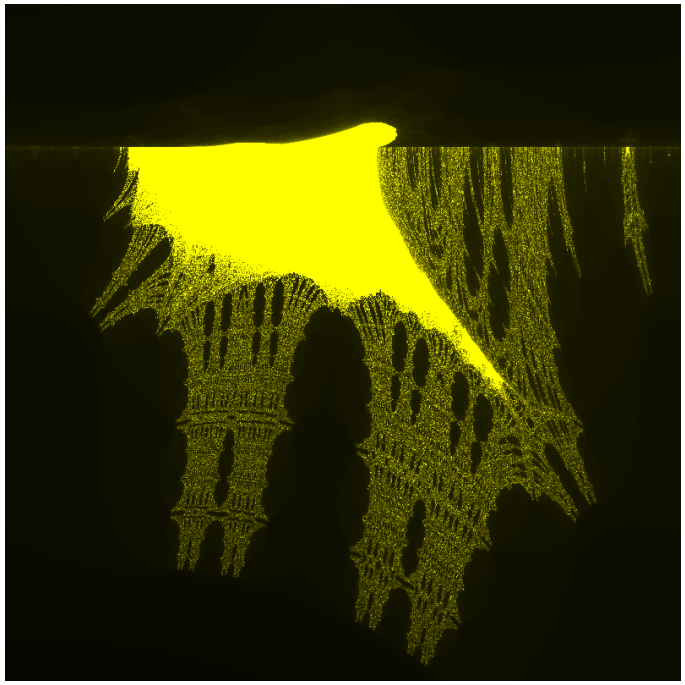


Figure – Fractale du *Burning-Ship*

SAÉ S3.A.01 - Développement d'une Application

Sprint 4 - Applications à d'autres suites complexes

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS